CS24301 — THEORY (3.0 CR) | MODULE II NOTES | BIT MESRA | NEP SCHEME

Module 2: Syntax Analysis

Context-Free Grammars, LL(1), SLR(1), CLR(1), LALR(1) & Parsing Conflicts

Module II: Syntax Analysis & Parsing Algorithms — Complete Syllabus Topics

1. Role of the Parser & Context-Free Grammars (CFG)
2. Ambiguity Elimination & Grammar Transformations
3. Left Recursion Elimination (Immediate & Indirect Algorithms)
4. Left Factoring Algorithm for Common Prefixes
5. FIRST and FOLLOW Sets Formal Formulations & Step-by-Step Traces
6. LL(1) Top-Down Predictive Parser Table & Stack Parsing Walkthrough
7. Bottom-Up Shift-Reduce Parsing & Handle Pruning
8. LR(0) Canonical Collection of Items & SLR(1) Parsing Tables
9. Canonical LR (CLR(1)) with Lookahead & LALR(1) State Merging
10. Parsing Conflicts (Shift/Reduce, Reduce/Reduce) & Error Recovery

Topic 1 & 2: Context-Free Grammars & Ambiguity

A **Context-Free Grammar (CFG)** is formally defined as a 4-tuple $G = (V, T, P, S)$:

V: Finite set of Non-Terminal characters (variables representing syntactic categories).
T: Finite set of Terminal characters (tokens returned by lexical analyzer).
P: Finite set of Production rules of the form $A \to \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.
$S \in V$: The designated Start Symbol.

Grammar Ambiguity:

A grammar G is **ambiguous** if there exists at least one string $w \in L(G)$ that has two or more distinct parse trees (or equivalently, two distinct leftmost derivations).

Classic example: The Dangling-Else Grammar:

$$\text{stmt} \to \textbf{if}\ \text{expr}\ \textbf{then}\ \text{stmt} \mid \textbf{if}\ \text{expr}\ \textbf{then}\ \text{stmt}\ \textbf{else}\ \text{stmt} \mid \textbf{other}$$

Resolved by disambiguating the grammar into `matched_stmt` and `unmatched_stmt`, matching each `else` with the closest unmatched `then`.

Topic 3 & 4: Grammar Transformations (Left Recursion & Left Factoring)

1. Immediate Left Recursion Elimination:

A production $A \to A\alpha \mid \beta$ (where β does not start with A) causes top-down parsers to loop infinitely. It is eliminated by introducing a new non-terminal A':

$$A \to \beta A', \qquad A' \to \alpha A' \mid \epsilon$$

2. General Left Recursion Elimination Algorithm:

Arrange non-terminals in some arbitrary ordering: $A_1, A_2, \ldots, A_n$.
For $i = 1$ to n:
For $j = 1$ to $i - 1$: Replace each production $A_i \to A_j\gamma$ with $A_i \to \delta_1\gamma \mid \delta_2\gamma \mid \cdots \mid \delta_k\gamma$, where $A_j \to \delta_1 \mid \delta_2 \mid \cdots \mid \delta_k$ are the current A_j productions.
Eliminate immediate left recursion among the A_i productions.

3. Left Factoring Algorithm (Removing Common Prefixes):

When two productions for A share a common prefix ($A \to \alpha\beta_1 \mid \alpha\beta_2$), the parser cannot determine which branch to choose based on one lookahead token. Left factoring extracts the common prefix:

$$A \to \alpha A', \qquad A' \to \beta_1 \mid \beta_2$$

Topic 5: Formal Formulations of FIRST and FOLLOW

Formal Mathematical Definitions

$\text{FIRST}(\alpha)$: Set of all terminals that begin strings derived from α. If $\alpha \xrightarrow{*} \epsilon$, then $\epsilon \in \text{FIRST}(\alpha)$.
$\text{FOLLOW}(A)$: Set of all terminals a that can appear immediately to the right of non-terminal A in some sentential form ($S \xrightarrow{*} \alpha A a \beta$). If A is the rightmost symbol in a derivation, the input end-marker \$ is in $\text{FOLLOW}(A)$.

Rules for Computing FIRST:

If X is a terminal, $\text{FIRST}(X) = \{X\}$.
If $X \to \epsilon$ is a production, add ϵ to $\text{FIRST}(X)$.
If $X \to Y_1 Y_2 \dots Y_k$ is a production:
Add all non-ϵ symbols from $\text{FIRST}(Y_1)$ to $\text{FIRST}(X)$.
If $\epsilon \in \text{FIRST}(Y_1)$, add all non-$\epsilon$ symbols from $\text{FIRST}(Y_2)$ to $\text{FIRST}(X)$, and so forth, until some Y_i does not contain ϵ.
If all $Y_1, \dots, Y_k$ contain ϵ, add ϵ to $\text{FIRST}(X)$.

Rules for Computing FOLLOW:

Place \$ in $\text{FOLLOW}(S)$, where S is the start symbol.
If there is a production $A \to \alpha B \beta$, everything in $\text{FIRST}(\beta)$ except ϵ is in $\text{FOLLOW}(B)$.
If there is a production $A \to \alpha B$, or $A \to \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ, everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

🏛️ Step-by-Step Solved Problem: Arithmetic Grammar FIRST & FOLLOW

Grammar (G):

- $E \to TE'$
- $E' \to +TE' \mid \epsilon$
- $T \to FT'$
- $T' \to *FT' \mid \epsilon$
- $F \to (E) \mid \mathbf{id}$

Computed Sets:

Non-Terminal	FIRST Set	FOLLOW Set
E	$\{(, \mathbf{id}\}$	$\{), \$\}$
E'	$\{+, \epsilon\}$	$\{), \$\}$
T	$\{(, \mathbf{id}\}$	$\{+,), \$\}$
T'	$\{*, \epsilon\}$	$\{+,), \$\}$
F	$\{(, \mathbf{id}\}$	$\{*, +,), \$\}$

Topic 6: Top-Down LL(1) Predictive Parsing

An **LL(1) Parser** scans input from Left-to-right, produces a Leftmost derivation, using 1 lookahead token.

> **LL(1) Grammatical Condition**
>
> A grammar G is $LL(1)$ if and only if for every pair of productions $A \to \alpha \mid \beta$:
> 1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$.
> 2. If $\epsilon \in \text{FIRST}(\alpha)$, then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$.

Complete LL(1) Parsing Table:

Non-Terminal	$\mathbf{id}$	$+$	$*$	$($	$)$	$\$$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to \mathbf{id}$			$F \to (E)$		

Topics 7 – 9: Bottom-Up LR Parsing Landscape

LR Parsers (Left-to-right scan, Rightmost derivation in reverse) are the most powerful deterministic shift-reduce parsing engines.

Parser Class	Item Structure	Reduce Action Decision	State Complexity & Power
1. LR(0)	$[A \to \alpha \cdot \beta]$	Reduce across all columns unconditionally.	Fewest states; weak; fails on almost all programming grammars.
2. SLR(1)	$[A \to \alpha \cdot \beta]$	Reduce $A \to \alpha$ only on input symbols $a \in \text{FOLLOW}(A)$.	Same number of states as LR(0); resolves many Shift/Reduce conflicts.
3. CLR(1)	$[A \to \alpha \cdot \beta,\ a]$ (with Lookahead a)	Reduce $A \to \alpha$ only on exact lookahead symbol a.	Most powerful deterministic class; massive number of states (thousands).
4. LALR(1)	Merged LR(1) items with identical core $LR(0)$ components.	Reduce on merged lookahead sets.	Same number of states as SLR(1); nearly the parsing power of CLR(1); used in Yacc/Bison.

🏛️ Top BIT Mesra Exam Questions & Answers (Module II)

Q1. Prove why an ambiguous grammar can never be LL(1) or LR(1). (8 Marks)

Proof: An ambiguous grammar generates at least two distinct parse trees for some string w.
1. In an **LL(1) parser**, this ambiguity manifests as multiple production entries in the same parsing table cell $M[A, a]$, violating the single-entry requirement of deterministic LL(1) tables.
2. In an **LR(1) parser**, the two distinct derivations create either a **Shift/Reduce conflict** (where the parser cannot decide whether to shift or reduce) or a **Reduce/Reduce conflict** (where the parser cannot decide which of two distinct productions to reduce by). Thus, ambiguous grammars can never be parsed deterministically by LL(1) or LR(1).

Q2. Explain the difference between SLR(1) and LALR(1) parsers. (6 Marks)

Both SLR(1) and LALR(1) parsers have the exact same number of states (identical to the LR(0) state machine). However, they differ in how they decide reduction actions:
- **SLR(1):** Places reduce actions for $A \to \alpha$ in all columns corresponding to $\text{FOLLOW}(A)$. $\text{FOLLOW}(A)$ is a global set that includes symbols that may never legally follow A in that specific context.
- **LALR(1):** Computes specific context-sensitive lookahead sets during item propagation, placing reduce actions only on valid lookaheads. As a result, LALR(1) eliminates many false Shift/Reduce and Reduce/Reduce conflicts present in SLR(1).